

```

        reward = rewards[states[current_state_idx]][2]
    # Randomly pick next state based on Markov probabilities
    next_state_idx = np.random.choice([0, 1], p=transition_probs)
    return next_state_idx, reward
# Simple Simulation
current_state = 0 # Start at "High"
total_reward = 0
print(f"Starting in state: {states[current_state]}")
for day in range(5):
    # Logic: If battery is low, recharge. Else, search.
    action = "recharge" if current_state == 1 else "search"
    next_state, reward = simulate (current_state, action)
    print (f"Day {day+1}: State={states[current_state]},
    Action={action}, Reward={reward}")
    current_state = next_state
    total_reward += reward
print (f"Total Reward Earned: {total_reward}")

```

The outputs are:

Starting in state: High

Day 1: State=High, Action=search, Reward=10

Day 2: State=Low, Action=recharge, Reward=0

Day 3: State=High, Action=search, Reward=10

Day 4: State=Low, Action=recharge, Reward=0

Day 5: State=High, Action=search, Reward=10

Total Reward Earned: 30

Key Markov properties in this code are:

- **Memoryless property:** The 'simulate step' function only cares about the *current_state_idx*. It doesn't care about the past performance, and the probability of the battery dropping remains a constant 0.3.
- **Transition matrix:** Since the model is an MDP, the *P_search* and *P_recharge* arrays are stochastic matrices (rows sum to 1.0).
- **State space:** The problem considers the complex robot states and transitions as finite.

Summary of logic and dataflow

- **State space:** The agent looks at its current battery conditions:
 - a. `states = [High, Low]`
- **Consult policy:** Based on the battery, it picks an action (Search or Recharge). This is defined using a dictionary, where each state [High, Low] has a list of rewards corresponding to the actions [Search, Wait, Recharge].

- a. `rewards = {`
- b. `"High": [10, 2, -1], # High reward for searching`
- c. `"Low": [-5, 1, 0] # Penalty (-5) if searching fails on low battery`
- d. `}`

-1 is to discourage the robot to recharge if the battery is already full and -5 is to discourage search when the battery is low.

- **Consult Markov matrix:** A Markov transition matrix defines the probability of moving from State A to State B given a specific action:

- a. `P_search = np. array ([`
- b. `[0.7, 0.3], # High -> High (70%), High -> Low (30%)`
- c. `[0.0, 1.0] # Low -> High (0%), Low -> Low (100%)`
- d. `])`

Search: There is a 30% chance it will transition to State 1 (low).

Recharge: There is a 100% chance that State 1 will remain in State 1 (low).

- **Receive reward:** The agent collects reward points based on the action-state pair (search/recharge, state). The function is *simulate_step (current_state_idx, action)*.
- **Update:** The *next_state* becomes the *current_state* for the next loop.
- **The decision logic (The policy):** In this simple example, the hardcoded policy is: 'If battery is low, recharge; otherwise, search'.

```

action = "recharge" if current_state == 1 else "search"

```

This Python code will simulate day-wise cleaning, charging and reward calculation for five days.

Reinforcement learning is based on machine learning, it acts based on the state and action of the problem domain. It is not data-driven and can function in an uncertain environment, whereas supervised and unsupervised learning are data-driven and operate in a known domain. **END** 🐧

 By: Neha Ray

The author is a senior systems engineer at Infosys. For the last three years she has been focusing on enterprise-grade systems and automation.